EchoSphere Digital Assignment 3

Software Engineering Project Report and Testing Evidence

Name KAILASH P Register Number 24BRS1382

Review 3

Project Overview

EchoSphere is a personal learning environment that organises study documents and notes and provides AI summaries, explanations, flashcards, exam questions, knowledge graphs and knowledge-base chat. The current release combines a React interface, an Express API, MongoDB persistence, LanceDB retrieval and local Ollama models.

Problem Statement

Students and researchers keep useful knowledge across separate documents and notes. Finding relevant passages, revising long materials and connecting related concepts takes time. EchoSphere brings these materials into one workspace and provides study outputs derived from selected documents or retrieved knowledge-base context.

Users and Objectives

The primary users, carried forward from DA1, are university students preparing for assessments and researchers reviewing a collection of papers. The objective is to reduce retrieval and revision effort while keeping the source material available for checking AI output.

Report Scope

This report presents the project requirements, system design, implementation and completed DA3 testing. The recorded results include 64 passing Jest tests, 33 passing Postman assertions across 22 live requests, a 92.28 percent mutation score for request validation, and successful frontend and Docker image builds. Automated tests, live API tests and selected browser workflows were verified separately.

Project repository and code

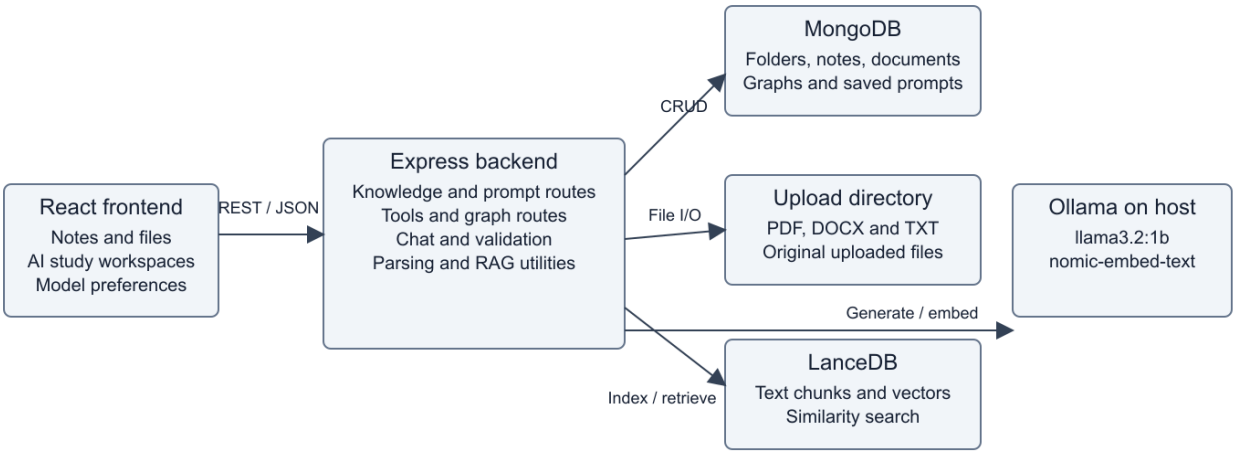
User Stories and Priorities

DA1 records 25 user stories as GitHub issues. The following table consolidates representative priorities for the current single-user release; local model configuration supersedes the earlier Gemini API-key requirement.

Priority	User story	Current scope
Must	As a learner, I want to upload study documents so I can work from my own materials.	PDF, DOCX and TXT ingestion.
Must	As a learner, I want extracted text so AI tools can process document content.	pdf-parse, mammoth and TXT reading.
Must	As a learner, I want notes stored across sessions so I can retrieve them later.	MongoDB note persistence.
Must	As a learner, I want summaries so I can revise long materials quickly.	Selected-document summarisation.
Must adapted	As a learner, I want to choose a local AI model so study tools use my preferred model.	Ollama selection; no Gemini key required by active routes.
Should	As a learner, I want folders so I can organise related documents.	Hierarchical folders and browsing.
Should	As a learner, I want adjustable output length and explanation depth.	Summary and explanation settings.
Should	As a learner, I want extracted concepts and relationships to see connections.	AI graph generation and persistence.
Should	As a learner, I want consistent navigation between workspaces.	Persistent React sidebar.
Could	As a learner, I want to inspect an interactive two-dimensional knowledge graph.	Force-directed graph view.
Could	As a learner, I want saved prompts and a theme preference.	Prompt workspace and light/dark mode.
Current extension	As a learner, I want flashcards, practice questions and knowledge-base chat.	Flashcards, exam predictor and RAG chat.
Outside release	Multi-user JWT login and OCR for scanned documents.	Remain outside the DA1 single-user scope.

Full user-story issue catalogue

System Architecture and System Design



Docker Compose builds frontend, backend and MongoDB; Ollama runs separately on the host.
Chat uses vector retrieval. Document tools read the selected document text directly.

Current architecture: MongoDB holds application records; uploaded files remain on disk; LanceDB stores indexed text chunks; Ollama provides local generation and embeddings.

The modular frontend and REST API separation follow the principles described in DA2. Knowledge routes manage folders, documents and notes; tools routes generate study outputs; graph and prompt routes persist their own records. RAG utilities handle chunking, embeddings and retrieval.

Uploaded files and notes are saved first, then vector ingestion runs in the background. Chat retrieves relevant LanceDB chunks; summarisation, explanation, flashcards, exam questions and graph generation read selected document text directly. Ollama is a host service outside the frontend/backend/MongoDB Compose services.

[Original DA2 architecture drawing on Google Drive]

Implementation and Module Responsibilities

Frontend and user interaction

React and TypeScript implement the dashboard, knowledge base and study workspaces. Tailwind CSS provides the layout and light/dark presentation. The persistent sidebar keeps navigation consistent with the DA2 wireframes. Each workspace holds its selection, loading, result and error states; HTTP requests connect it to Express. The summary workspace displays source text beside generated output so the learner can compare them.

Knowledge ingestion and storage

A learner creates a folder, uploads a supported file or writes a note. Multer receives uploads with a 50 MB limit. PDF text is extracted using pdf-parse, DOCX text using mammoth, and TXT files are read as text. MongoDB stores metadata and notes, while original uploads remain in the configured upload directory. Web clipping uses fetch and cheerio to extract readable page content and save it as a note; success depends on network access and page structure.

Generation and retrieval

Document tools obtain the selected record and extract its text before calling Ollama. The concept explainer also accepts a topic without a document. Flashcards, exam questions and graphs request structured JSON so their outputs can be rendered as interactive components. Chat follows a separate RAG path: text is split into overlapping chunks, nomic-embed-text creates embeddings, LanceDB retrieves relevant chunks, and the selected generation model composes an answer from the retrieved context.

Module	Main responsibility
knowledge.js	Folder, file, note, download and web-clipping operations.
tools.js	Summary, explanation, flashcards and exam-question generation.
graph.js and prompts.js	Graph generation/persistence and reusable prompt storage.
rag.js and validation.js	Chunking, embeddings, retrieval, shared generation and request validation.

The generation model used in the recorded demo is llama3.2:1b. Local Ollama calls do not require a Gemini API key. Model selection and the theme preference are stored in the browser; application records remain in MongoDB.

Design Principles and Report Continuity

Principle from DA2	Application in the current release
Modularity and cohesion	React workspaces and dedicated route modules separate notes, graphs, prompts and AI tools.
Loose coupling	The frontend exchanges HTTP/JSON with Express. Persistence and model generation sit behind backend routes.
Abstraction	Learners select documents and study settings; backend utilities handle parsing, chunking and vector retrieval.
Usability	The persistent sidebar, familiar file browser, source/summary view and graph workspace preserve the Figma design direction.
Maintainability	Shared validation, configurable database/model addresses, repeatable tests and Docker builds support diagnosis and change.

Changes Carried into DA3

Earlier description	DA3 description
Gemini and browser API-key storage	Active AI routes use Ollama. Browser storage holds the selected model preference.
PDF DOCX TXT MD upload labels	The current backend accepts PDF, DOCX and TXT; Markdown display support does not establish MD ingestion support.
All study tools use RAG	Chat uses vector retrieval. Document tools use the selected document text directly.
Synchronous processing before persistence	Records are saved and acknowledged before background vector ingestion.
Everything inside Docker Compose	Frontend, backend and MongoDB are Compose services; Ollama runs separately on the host.
Three to five seconds and near-zero hallucinations	These were DA1 targets. The live run averaged 7.6 seconds and reached 59.8 seconds; factual accuracy, uptime and retention are not established by these tests.

Design of Tests

The repeatable suite combines Express and real MongoDB integration with focused validation regression tests. AI calls are stubbed in Jest for repeatability. The separate Postman collection uses real MongoDB and Ollama; browser checks verify that a saved note and generated flashcards are displayed.

Case	Action or input	Expected result	Recorded evidence
TC01	GET /api/health	HTTP 200 and connected database	Jest and Postman pass.
TC02	Create, browse and rename a folder	Created record appears with the updated name	Jest and Postman pass.
TC03	Create note, edit content and reindex	Content persists and index refresh is invoked	Jest pass; creation also verified in browser.
TC04	Upload TXT, download it, list DOCX metadata	TXT is extracted; files are visible and downloadable	Jest pass. PDF/DOCX parser accuracy is not comprehensively measured.
TC05	Blank fields, unsupported upload, invalid IDs	HTTP 400 JSON error	Jest and applicable Postman checks pass.
TC06	Create, list and delete a saved prompt	Correct persistence and deletion	Jest pass; create/list also pass in Postman.
TC07	Save graph and retrieve metadata	Nodes and metadata are retained	Jest and Postman pass.
TC08	Generate summary, explanation, cards, exam, graph and chat	Non-empty outputs with expected response shapes	Six live AI flows pass in Postman; five cards displayed in browser.
TC09	Delete folder; invoke clear-all in test database	Contents preserved at root; isolated data cleared	Jest pass; no production database is used.
TC10	Change validation conditions, strings and boundaries	Regression tests detect behavioral changes	227 killed, 19 survived, zero uncovered mutations.

Test-only storage uses MongoDB on port 27018 and separate upload/vector directories. Response-shape tests do not prove the factual correctness of every AI answer.

Test Execution Method and Acceptance Criteria

Repeatable automated checks

Jest runs the backend suite in one worker and Supertest sends requests directly to the exported Express application. The integration tests connect to a real MongoDB instance at 127.0.0.1:27018 using the echosphere_jest database. AI generation and indexing are stubbed in this suite so failures can be attributed to application behavior rather than varying model output. The test upload directory is artifacts/jest-uploads; the suite drops its disposable database after execution.

Live integration checks

The Postman collection runs in order because later requests reuse identifiers created by earlier requests. It creates a folder and note, saves a prompt, calls the live study tools, saves/retrieves a graph, updates a note and exercises invalid inputs. A request passes only when its assertions pass: for example, a folder must return HTTP 201, appear in the subsequent list, and provide an identifier for dependent operations. MongoDB and Ollama are real services in this run.

Expected results and negative cases

A successful summary or explanation must return HTTP 200 with non-empty result text. Flashcards must contain question/answer fields, exam responses must contain questions, and graphs must contain nodes and links. Blank names, missing document selections, invalid counts and malformed chat history should return HTTP 400 with JSON error information. Unknown API paths should return JSON 404. These checks establish response behavior; they do not grade the educational correctness of every generated statement.

Regression and mutation checks

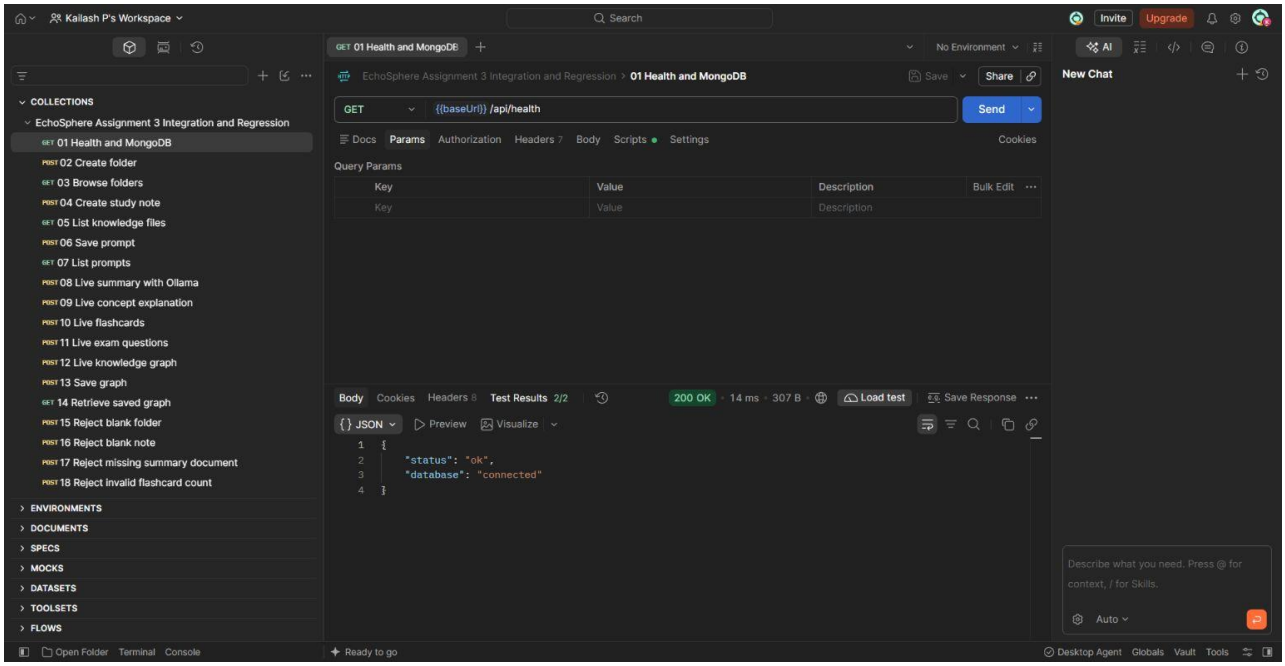
Regression tests repeat the expected workflows after fixes to detect unintended changes. Stryker then mutates backend/validation.js and reruns the tests. A killed mutant means the tests detected a changed behavior; a surviving mutant identifies a possible test weakness or equivalent behavior requiring review. This gives additional evidence about the validation tests, rather than measuring the whole application.

Reproduction commands

With the disposable MongoDB service available, run npm test and npm run test:mutation from backend. Import EchoSphere.postman_collection.json into Postman and run its requests in order against the configured local backend. Use the Desktop Agent for Postman web because the Cloud Agent cannot reach localhost. TESTING.md records the isolated service configuration and the original run instructions.

The recorded automated results comprise 64 passing Jest tests, 33 passing Postman assertions and a 92.28 percent mutation score for request validation. Browser checks provide additional evidence for selected user workflows; they do not replace the automated suite or establish exhaustive coverage of the final application.

Integration Testing



Postman returns HTTP 200 and shows both health checks passing, confirming an available Express API and connected MongoDB database.

Integration Testing with Live AI



The recorded live Postman CLI execution passes all 33 assertions across 22 requests against MongoDB and Ollama.

This screenshot displays the saved CLI result. The previous page is captured from the actual Postman web interface.

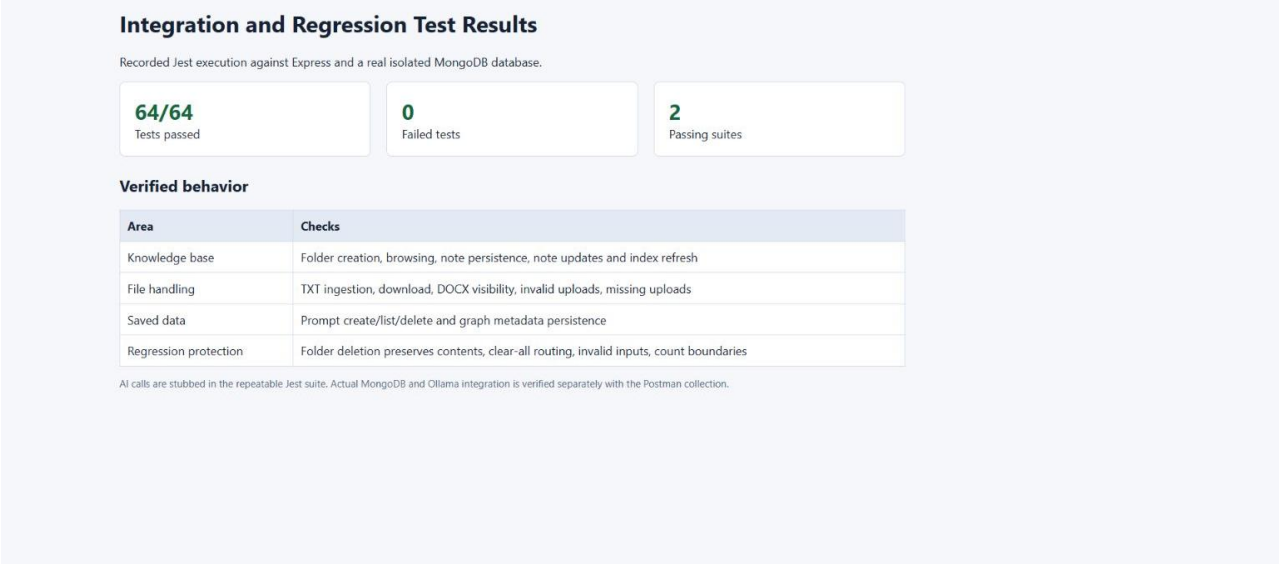
What the integration evidence demonstrates

The health request checks that Express responds and MongoDB is connected. The full collection checks a longer chain: create data, list it, use it in a tool and retrieve saved output. This is important because a successful model response alone would not demonstrate correct storage or document selection.

How the output was obtained

Postman sends HTTP requests to the local API, evaluates the pm.test assertions and records statuses and response times. The health image is from the actual Postman web interface. The collection-results image is a viewer of the saved CLI log; it is labelled separately and should not be described as a screenshot of the Postman application interface.

Regression Testing



Integration and Regression Test Results

Recorded Jest execution against Express and a real isolated MongoDB database.

64/64	0	2
Tests passed	Failed tests	Passing suites

Verified behavior

Area	Checks
Knowledge base	Folder creation, browsing, note persistence, note updates and index refresh
File handling	TXT ingestion, download, DOCX visibility, invalid uploads, missing uploads
Saved data	Prompt create/list/delete and graph metadata persistence
Regression protection	Folder deletion preserves contents, clear-all routing, invalid inputs, count boundaries

AI calls are stubbed in the repeatable Jest suite. Actual MongoDB and Ollama integration is verified separately with the Postman collection.

Jest passes 64 tests covering persistence, upload handling, validation and corrected workflows.

The integration suite uses real MongoDB and stubs AI calls; actual AI integration is exercised separately by Postman.

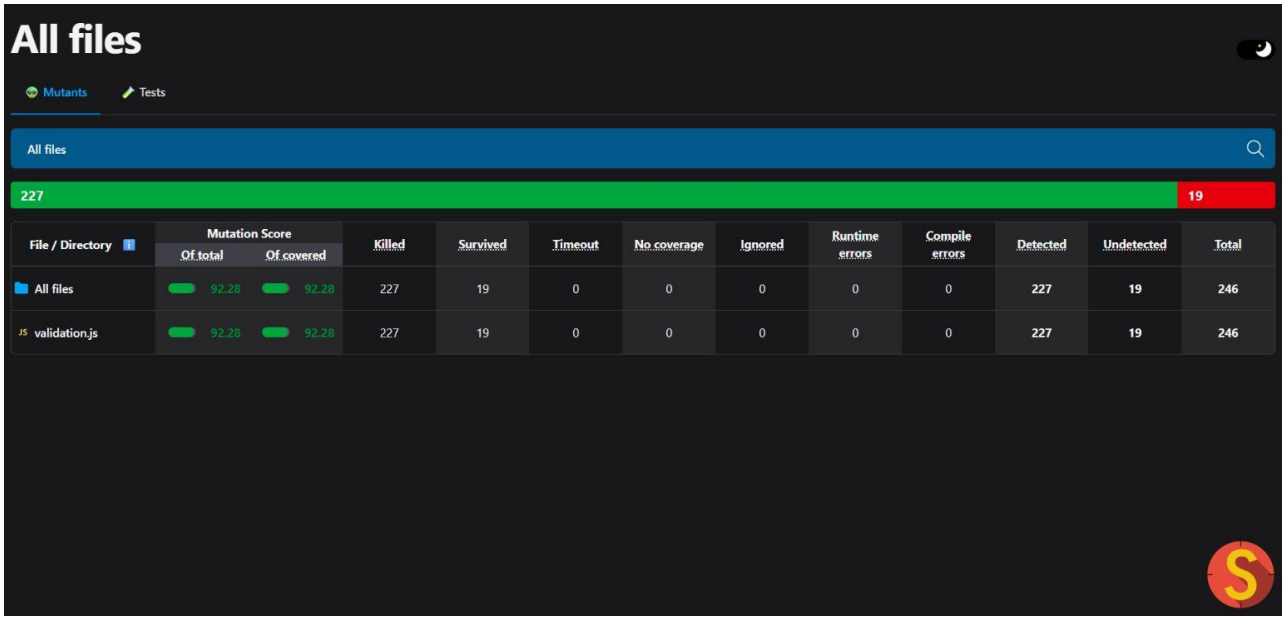
Regression coverage

The suite checks folder creation and browsing, note persistence and editing, TXT extraction and download, supported file listing, saved-prompt round trips, graph metadata preservation and error responses. It also verifies that editing a note invokes index refresh and that the clear-all request reaches the dedicated route in the disposable test setup.

Example of a meaningful assertion

After creating a note, the test reads the MongoDB record and checks its content. After editing the note, it verifies the indexing helper is called with the updated text. This detects a defect that a status-code-only check could miss: returning success while leaving stored content or the search index stale.

Mutation Testing



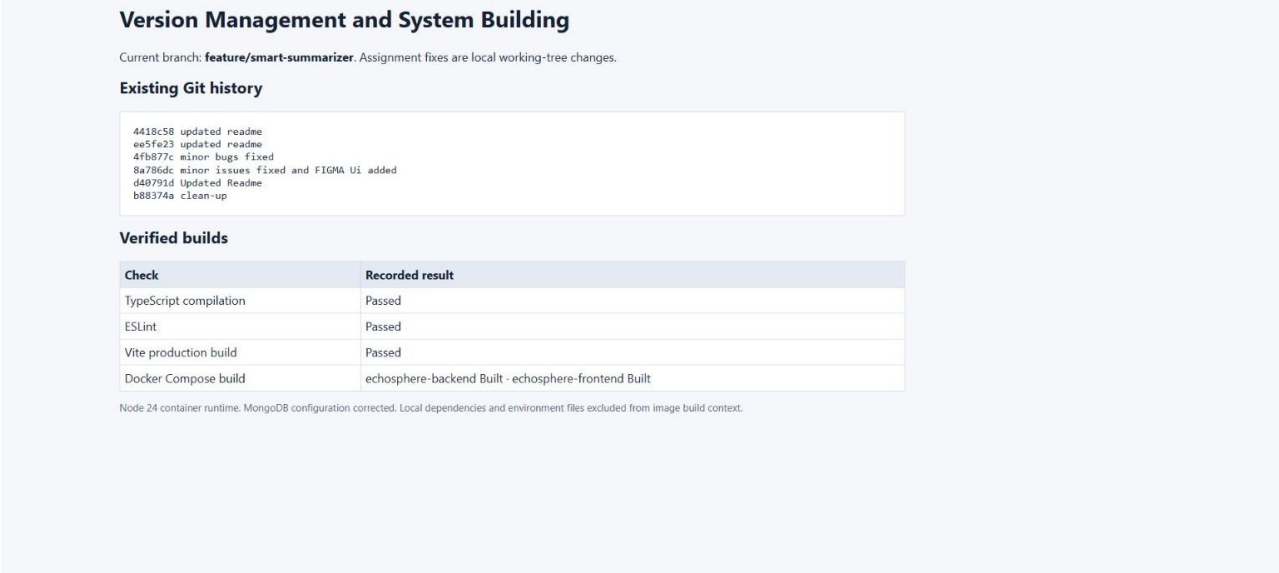
Stryker detects 227 of 246 mutations in request validation, producing a 92.28 percent score.

The scope is backend/validation.js. Nineteen mutations survive; this is not a project-wide score.

Interpretation of the score

The recorded score is 227 divided by 246, multiplied by 100, giving 92.28 percent. All recorded mutants were covered, but 19 survived. Consequently, the validation tests detected most introduced changes while leaving cases for further investigation. A high mutation score does not prove that all business logic, AI behavior or UI behavior is correct.

Version Management and System Building



Version Management and System Building

Current branch: **feature/smart-summarizer**. Assignment fixes are local working-tree changes.

Existing Git history

```
4418c58 updated readme
ee5fe23 updated readme
4fb877c minor bugs fixed
8a786dc minor issues fixed and FIGMA Ui added
d40791d Updated Readme
b88374a clean-up
```

Verified builds

Check	Recorded result
TypeScript compilation	Passed
ESLint	Passed
Vite production build	Passed
Docker Compose build	echosphere-backend Built · echosphere-frontend Built

Node 24 container runtime. MongoDB configuration corrected. Local dependencies and environment files excluded from image build context.

Git history is recorded and TypeScript, ESLint, Vite and both Docker image builds pass.

Git was used to manage the feature/smart-summarizer branch and maintain project history. DA2 documents the smart-summarizer pull request merged into main. The implementation includes shared Ollama generation handling and improved graph prompts. The build evidence demonstrates successful frontend checks and container packaging.

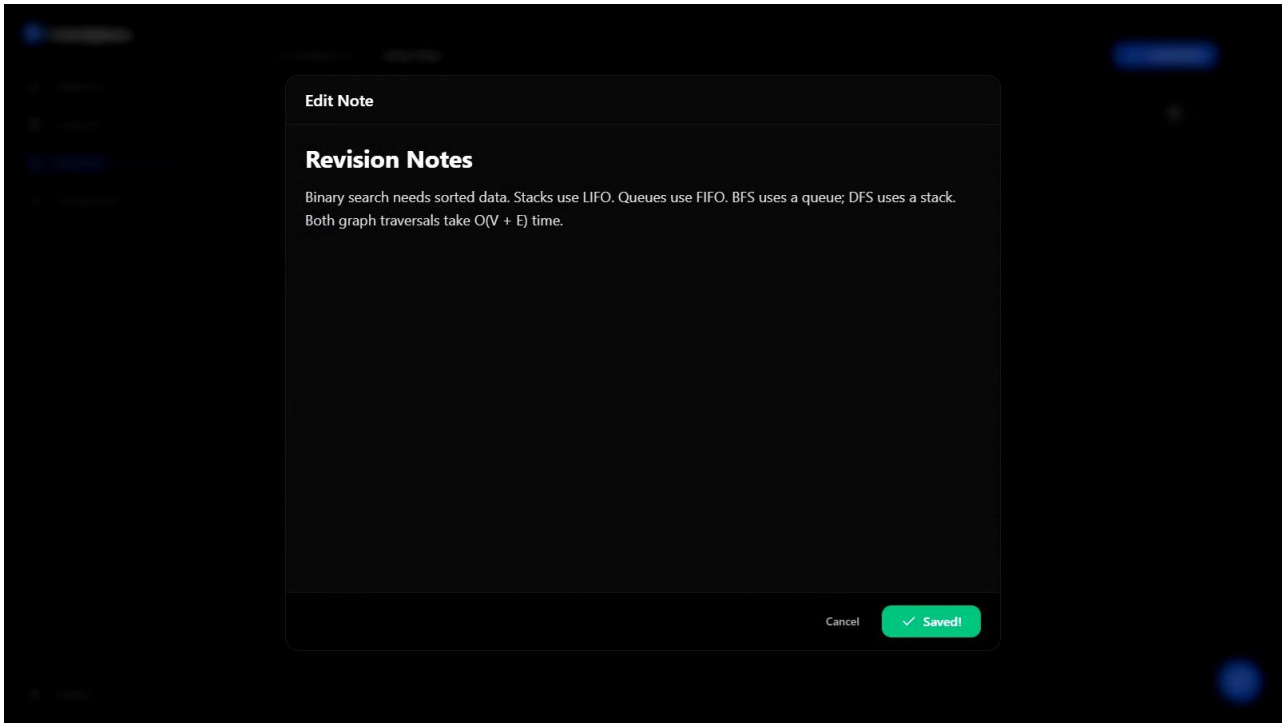
Build pipeline

TypeScript checks frontend types, ESLint checks configured code-quality rules and Vite produces the production frontend bundle. Docker image builds verify that the frontend and backend can be packaged with their declared dependencies. Successful builds do not by themselves establish deployment health; MongoDB, upload storage and the host Ollama service still need correct runtime configuration.

Version management evidence

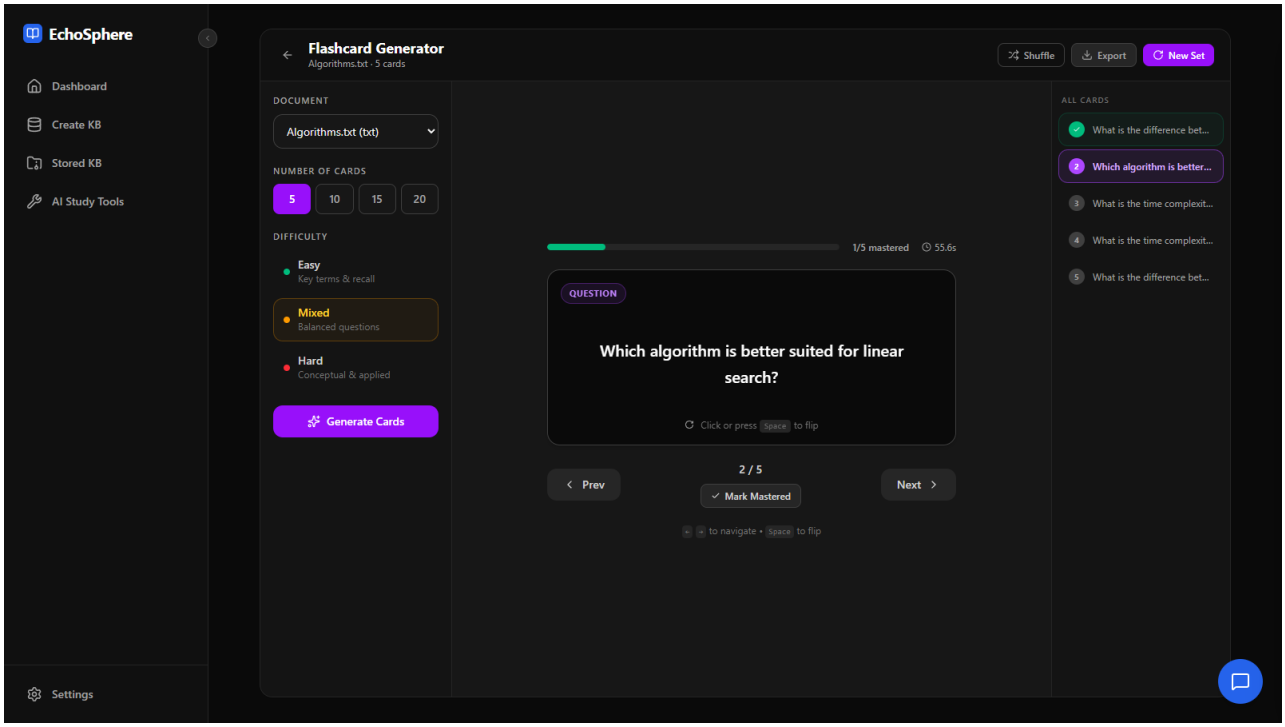
Git records the feature branch and commit history, supporting comparison of changes and rollback. DA2 documents the smart-summarizer pull request. The screenshots provide source-control and build evidence; they do not establish that every local change was pushed or that the application was deployed.

Developed Functionality Knowledge Base



Revision Notes was saved in Study Notes and its content was edited through the application interface.

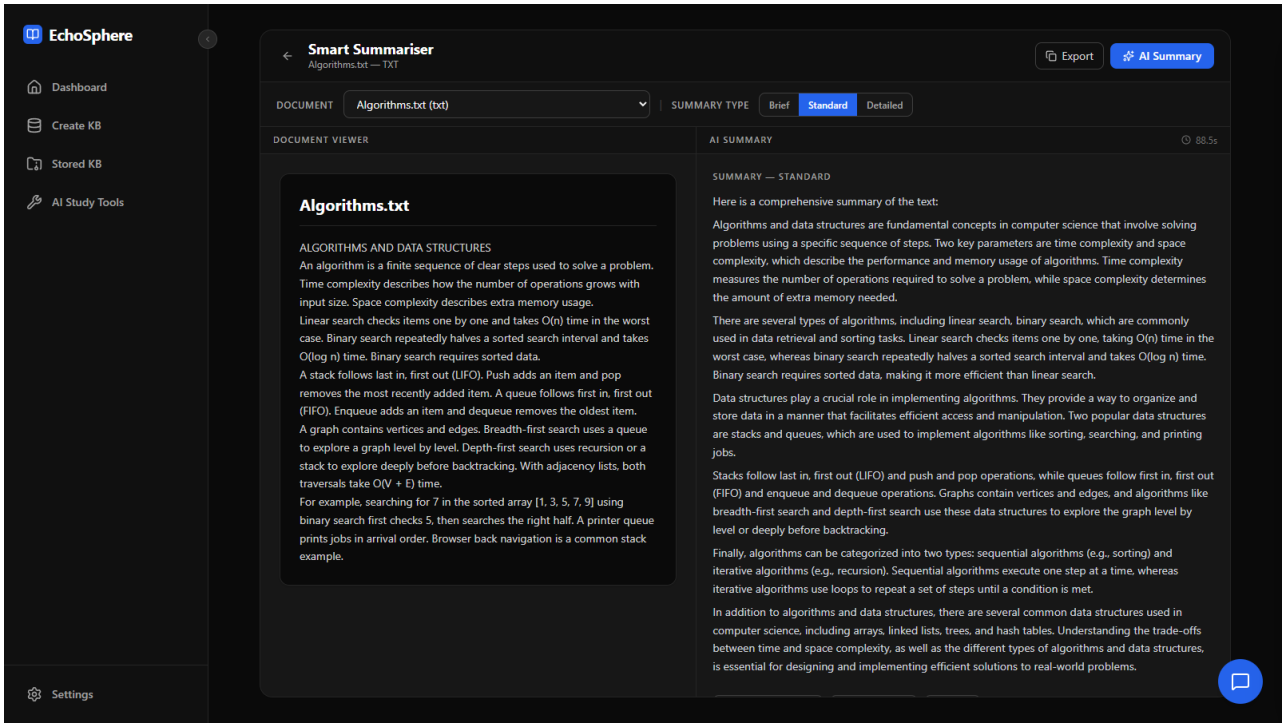
Developed Functionality Flashcards



Ollama generates five cards from Algorithms.txt; the browser shows navigation, mastery progress and the export control.

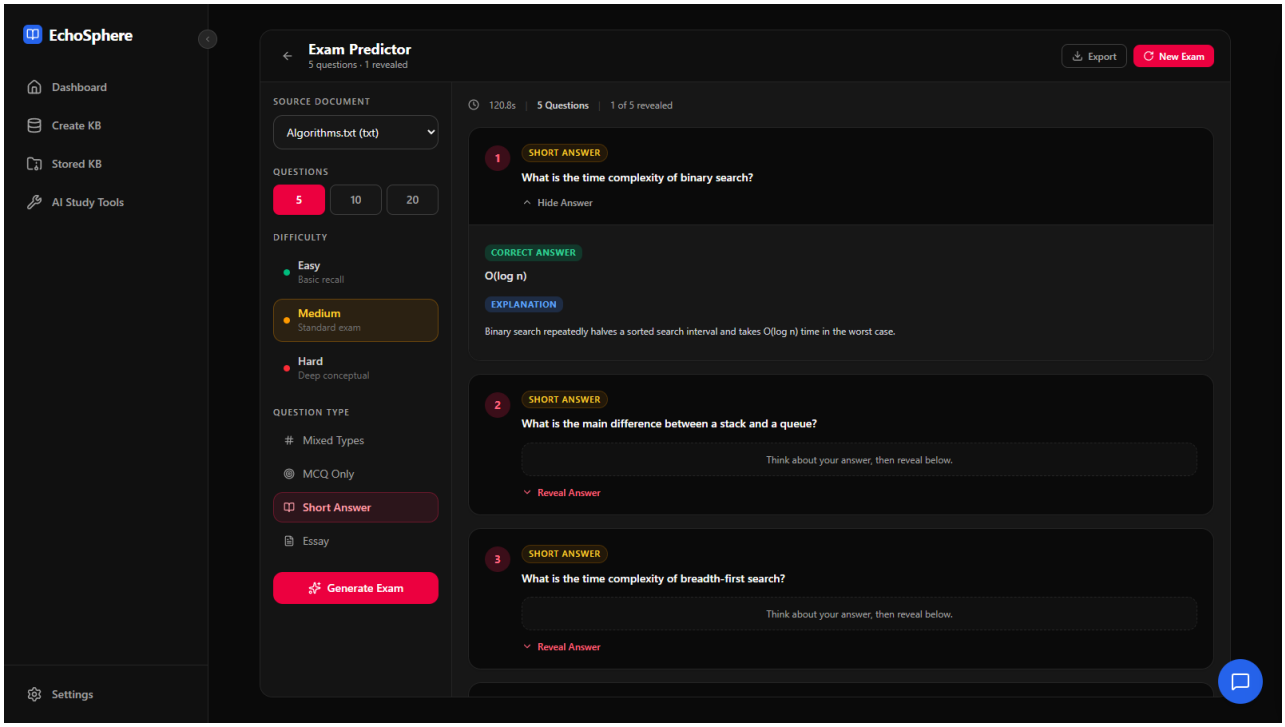
Additional Functionality Evidence

Document Summary



Algorithms.txt is shown beside the summary returned by the local model.

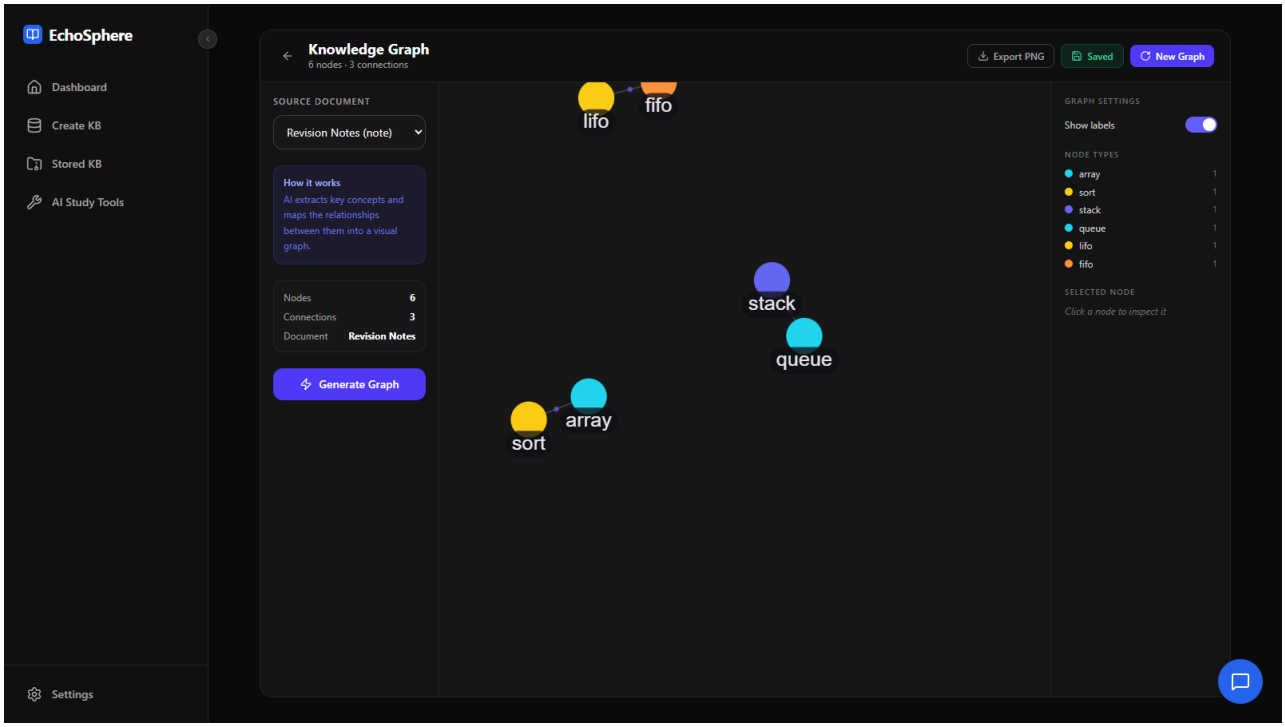
Practice Questions



Five short-answer questions are generated and the first answer is revealed.

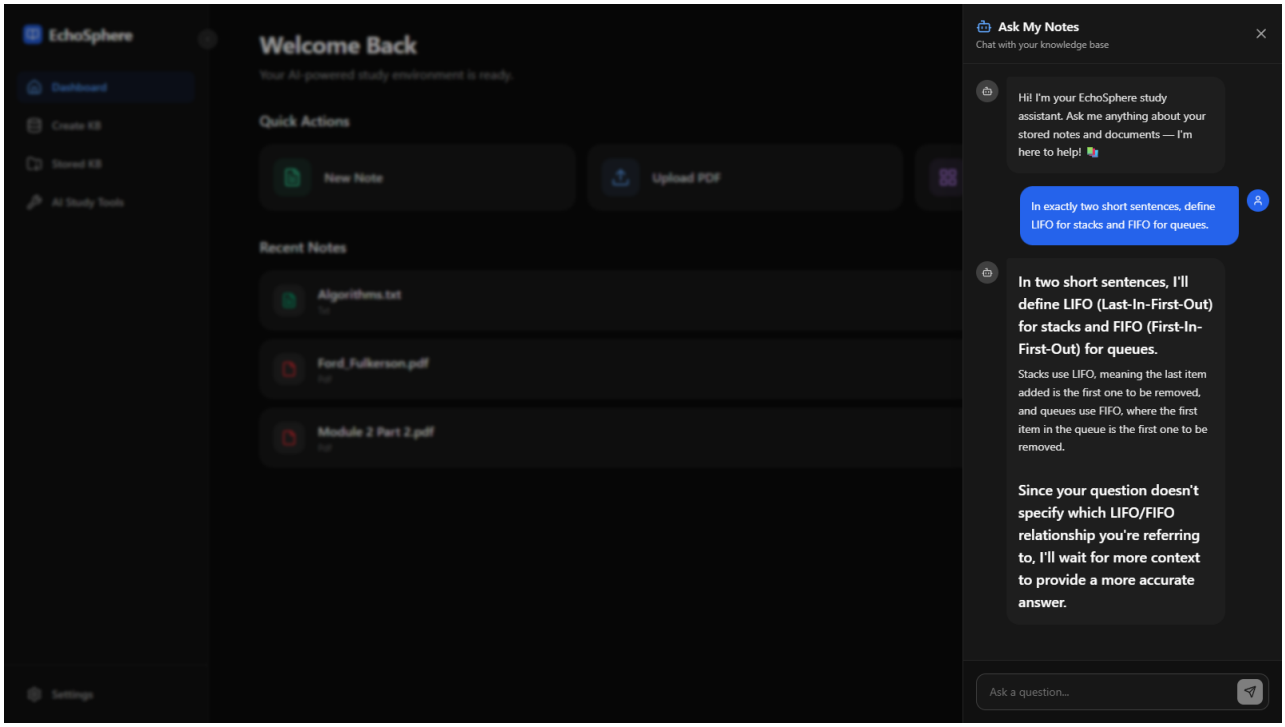
Graph and Chat Evidence

Saved Knowledge Graph



A knowledge graph was generated with concept labels and saved for the selected note.

Knowledge Base Chat



The question and the model response appear in the chat panel; factual accuracy remains subject to source checking.

Appendix A Software Requirements Specification

Purpose: provide a single-user local workspace for storing personal study materials and creating AI-assisted revision outputs. The requirements below describe the current release and its intended operation.

ID	Functional requirement
FR01	Accept PDF, DOCX and TXT uploads and store their metadata and original files.
FR02	Create, edit, list and delete notes; organise records within hierarchical folders.
FR03	Extract readable text and perform background chunking, embedding and vector indexing.
FR04	Generate selected-document summaries, explanations, flashcards, exam questions and graphs.
FR05	Persist and retrieve graph nodes/links and saved prompts.
FR06	Answer knowledge-base chat queries using retrieved context where available.
FR07	Allow local model selection, theme preference and consistent workspace navigation.
FR08	Validate supported request inputs and return clear JSON errors for rejected operations.

Constraints and Nonfunctional Requirements

The system requires local MongoDB and Ollama with the generation and embedding models installed. File uploads are limited to 50 MB. OCR, multi-user authentication and performance prediction are outside the implemented release. Existing automated tests cover specified behaviors; they do not establish security for public deployment, arbitrary-document parsing accuracy or guaranteed hallucination-free answers.

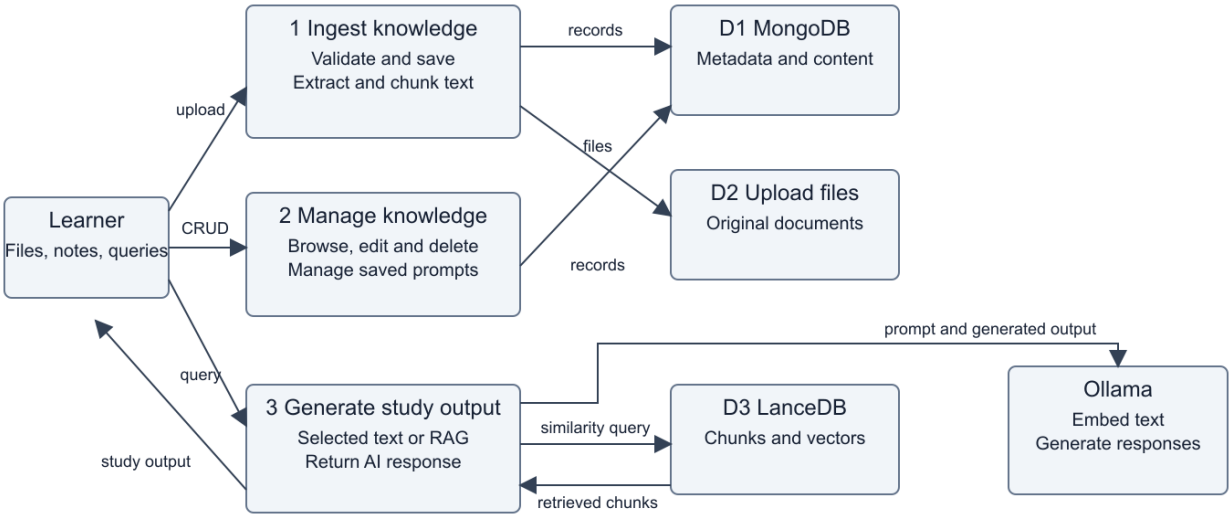
Tools and Technologies

React 19, TypeScript, Vite, Tailwind CSS, Express 5, Node.js 24, MongoDB/Mongoose, LanceDB/Apache Arrow, pdf-parse, mammoth, cheerio, Ollama, react-force-graph-2d, Git, Docker Compose, Postman, Jest, Supertest and Stryker.

Quality requirements and current status

Usability is supported by consistent navigation, visible loading states and source/output views. Maintainability is supported by route modules, shared validation, environment configuration and saved tests. Reliability is assessed through the recorded request and persistence checks. The original speed, uptime and near-zero hallucination goals remain targets: no load test, long-duration availability measurement or systematic factual-accuracy benchmark is included.

Appendix B Data Flow Diagram



Asynchronous ingestion sends extracted chunks for embedding before indexing in D3.

The DFD summarises learner inputs, knowledge ingestion, record management and study-output generation, with MongoDB, file storage and LanceDB as data stores.

Ingestion validates and saves input, extracts text and indexes embedded chunks asynchronously. Study tools obtain selected document text or retrieved context and call Ollama. Web clipping fetches a supplied web page and stores readable content as a note; this workflow requires network access.

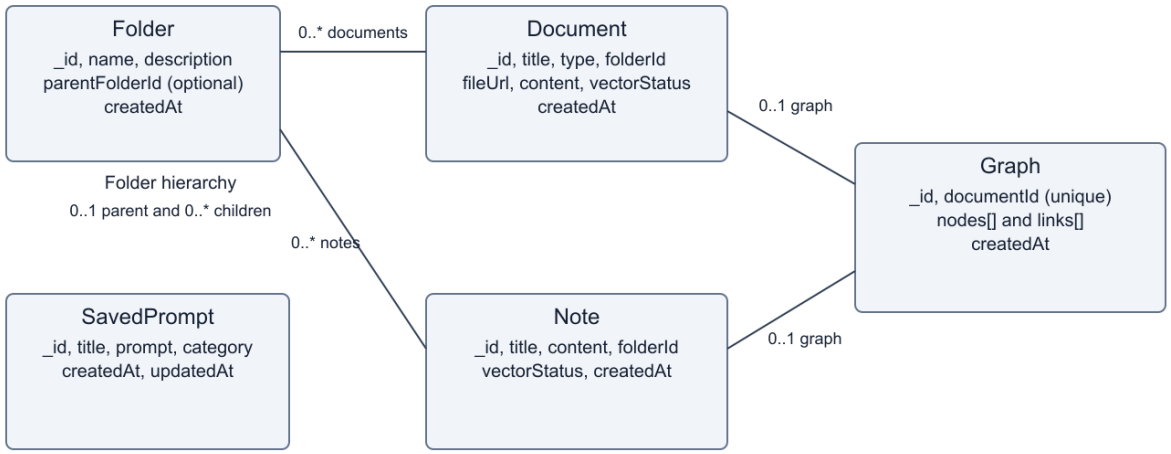
Inputs and outputs

The learner supplies a file, note, topic or question. Ingestion produces stored records, original files and vector chunks. Study generation produces text, structured cards/questions or graph nodes and relationships. Only chat depends on retrieved vector context; selected-document tools read source text directly.

Failure and timing considerations

A stored note can appear before its background vector ingestion completes. Therefore, immediate chat availability must not be inferred from a successful save response. Missing files, unsupported formats, unavailable models and generation errors are distinct failure paths. The UI should show the returned error while preserving the learner selection so it can be retried.

Appendix C Entity Relationship Diagram



Graph.documentId logically identifies a Document or Note; MongoDB does not enforce a foreign key.
SavedPrompt is independent. LanceDB chunks use documentId to associate vectors with stored content.

Logical relationships reflect the current MongoDB schemas. A document or note can belong to one optional folder, while a folder can contain many records.

Folder.parentFolderId models nesting. Graph.documentId is unique and logically identifies a document or note, although MongoDB does not enforce a foreign key. Nodes and links are embedded in Graph; SavedPrompt is independent. Vector chunks reside in LanceDB and carry documentId associations.

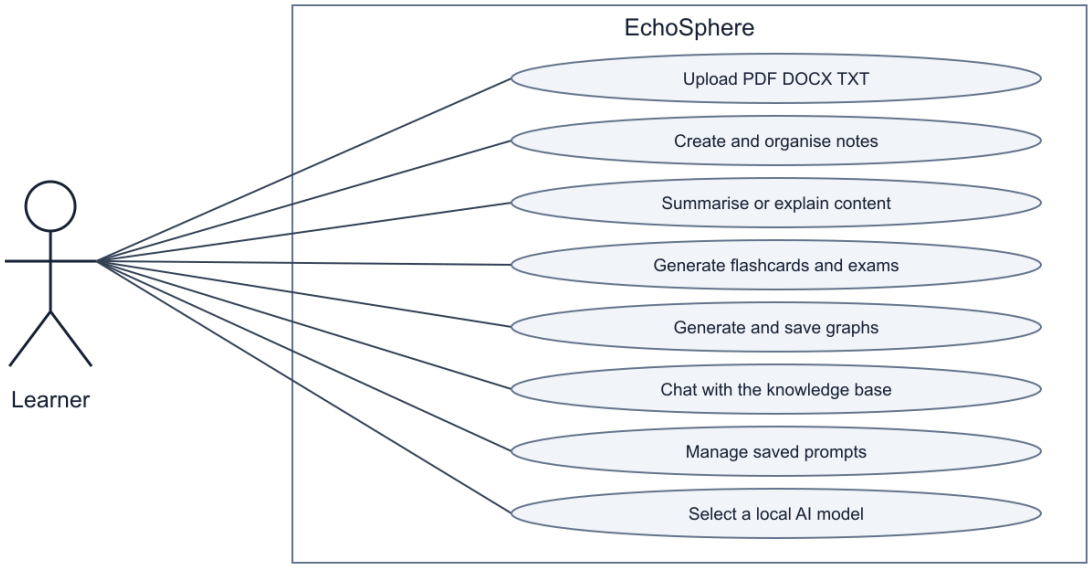
Record responsibilities

Document records identify uploaded source material and its type; Note records contain editable text, including the current web-clipping output. Folder records provide names and optional parent links. A Graph embeds nodes and links for its source document identifier. SavedPrompt records hold reusable prompt text and category information. LanceDB chunks associate vector content with the originating document identifier for retrieval.

Consistency considerations

The relationships are logical rather than enforced relational foreign keys. Application routes must handle deletion and movement consistently, retain graph metadata and refresh note vectors when content changes. The automated suite checks selected lifecycle cases, but it does not establish correctness for every possible interrupted indexing or concurrent-update scenario.

Appendix D UML Use Case Diagram



The learner interacts with document management, study tools, knowledge-base chat, saved prompts and local model configuration.

The single-user actor follows the DA1 scope. Multi-user JWT login and image OCR are excluded from this release. Flashcards, exam questions and chat extend the original summary, explanation and graph workspaces.

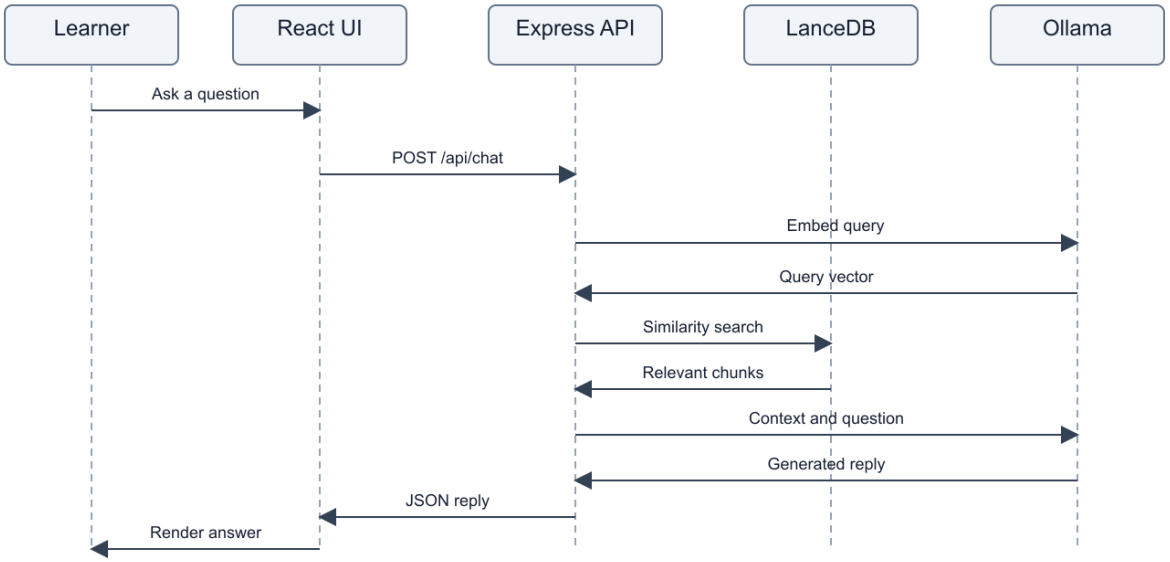
Example use case

To revise a document, the learner uploads it, confirms that it appears in the knowledge base, selects it in a study workspace and requests an output. The system reads its text, calls the chosen model and renders the result. The learner can then check the source, flip generated flashcards, reveal an exam answer or save the generated graph.

Preconditions and exceptions

The backend and MongoDB must be reachable, a supported source must be stored, and Ollama must have suitable generation/embedding models available. If a document is missing or input is invalid, the API should reject the request. If model loading fails, the error must be shown rather than presenting an empty output as successful completion.

Appendix E UML Sequence Diagram



The sequence shows the successful knowledge-base chat path from the browser request through query embedding, vector retrieval and response generation.

The main chat request is POST /api/chat. If no relevant vector chunks are returned, the implementation falls back to document titles or an empty-knowledge-base message. That fallback does not establish the contents of a document; users should check generated answers against their sources.

Sequence responsibilities

The browser submits a question and bounded chat history to Express. The backend embeds the query, searches the vector store, builds a context prompt and requests generation. The completed answer is returned as JSON and appended to the conversation.

References and Project Links

The prior submissions establish the project goals, user-story priorities, Figma design direction and architecture rationale. The links below preserve those references and provide access to the project code.

GitHub repository and code listing

Vision document and project README

User stories as GitHub issues

Figma prototype from DA2

Draw.io architecture file on Google Drive

Supporting Evidence

The report includes the project requirements, design diagrams, implementation details, testing methods and result screenshots. The online references provide access to the repository, user stories and design resources.